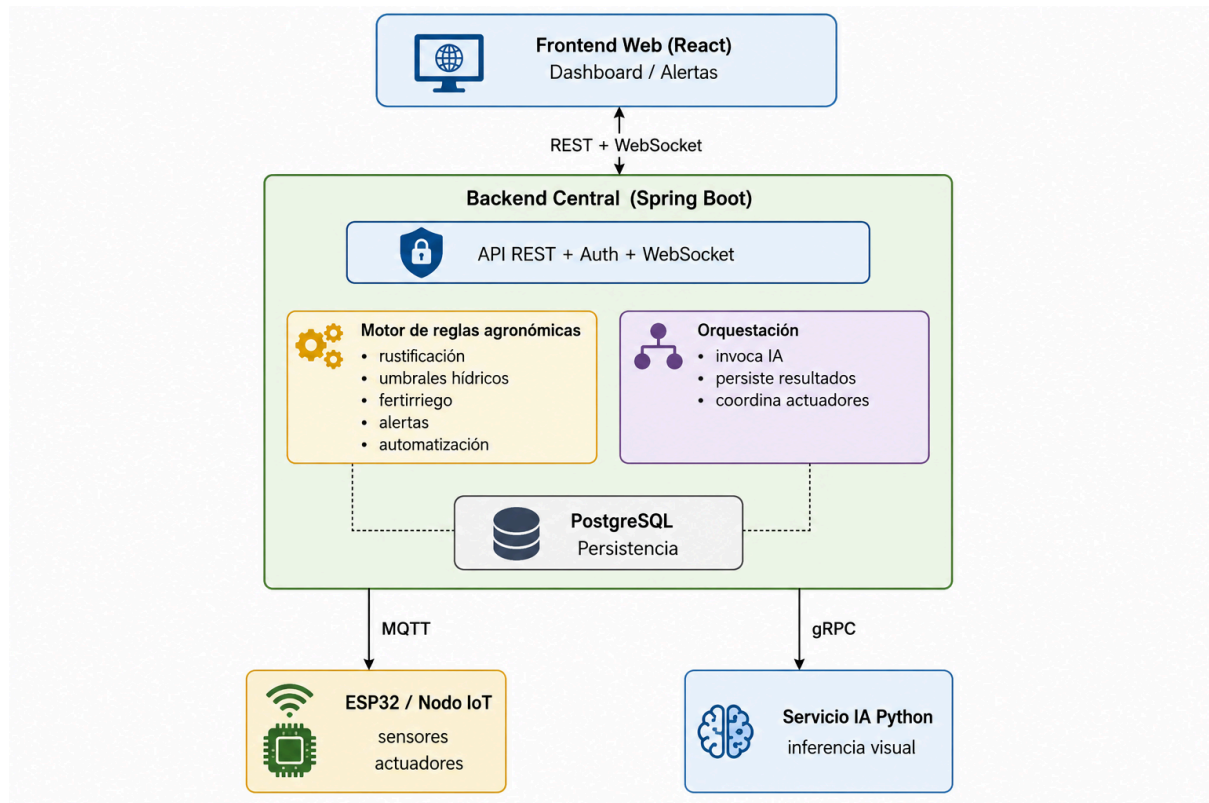


Arquitectura:



Por que cada protocolo de comunicacion?

REST + WebSocket: Lo que se usa para comunicar front con back, con websocket podemos tener el dashboard actualizado siempre sin necesidad de tener que hacerle refresh a la pagina

MQTT: Especifico para IoT, resiliencia ante conectividad inestable, mas liviano que otros protocolos de comunicacion.

gRPC: contratos fuertemente tipados, alto rendimiento, menos latencia, menos ancho de banda (ideal para nosotros que no sabemos que tan buena va a ser la conexion), especifico para back-to-back, y usar esta es chiche puro.

Estrategia de desarrollo: **contract-first + mock-driven development**.

Lo que yo pienso que tenemos que hacer (mas que nada con la info que tenemos hoy) es empezar por el back y el front, creando dumps que funcionen a modo de mocks para las respuestas que vamos a recibir del modelo y del nodo IoT.

Primero definimos los contratos de comunicacion ej:

1. Contrato del servicio IoT

JSON

```
{
  "nodeId": "zone-03",
  "timestamp": "2026-05-26T15:45:00Z",
  "sectorGroup": "macro-zone-3",
  "soil": {
    "humidity": 42.1,
    "temperature": 23.4,
    "ph": 6.1,
    "ec": 1.2,
    "nitrogen": 41,
    "phosphorus": 23,
    "potassium": 31
  },
  "air": {
    "temperature": 29.7,
    "humidity": 71.5
  },
  "light": {
    "lux": 32000,
    "uvIndex": 8.2
  },
  "deviceStatus": {
    "battery": 82,
    "online": true,
    "signal": -64
  }
}
```

2. Contrato IA

JSON

```
{
  "scanId": "scan-98342",
  "sectorId": "sector-314",
  "timestamp": "2026-05-26T15:46:10Z",
  "imageId": "gantry-sector314-img01.jpg",
  "predictions": [
    {
      "class": "CHLOROSIS",

```

```

    "confidence": 0.92,
    "severity": "MEDIUM",
    "boundingBox": null
  }
],
"healthScore": 61,
"recommendedAction": "FERTIGATION",
"modelVersion": "yolo-v1.2"
}

```

3. Contrato de decisiones (motor agronómico)

```

JSON
{
  "decisionId": "dec-001",
  "sectorId": "sector-314",
  "triggeredBy": [
    "sensor_humidity",
    "vision_ai",
    "weather_api"
  ],
  "reason": "chlorosis_detected",
  "action": "FERTIGATION",
  "confidence": 0.91,
  "executed": true,
  "timestamp": "2026-05-26T15:48:00Z"
}

```

Despues de esto deberiamos crearlos dumps, hacemos datasets gigantes con estos formatos.

Despues deberiamos simular escenarios ej:

Escenario 1 — estrés hídrico

12:00 humedad 48%
 13:00 humedad 43%
 14:00 humedad 39%
 14:05 IA → leaf curling
 14:06 backend → activar microaspersión
 14:15 humedad 57%

14:20 estado healthy

Escenario 2 — clorosis

N baja

IA detecta amarillamiento

backend activa fertirriego

48h después mejora score

Una vez que tenemos todo esto, podemos empezar a codear el backend y el frontend. Cuando tenemos el modelo y el servicio de IoT funcionando es tan simple como cambiar el lugar donde se consumen los datos..

opinen como lo harian ustedes, si les parece bien. Asi lo hacemos nosotros en el laburo y facilita bastante toda la etapa de desarrollo.. Mas que nada esta parte que mayormente la vamos a vibecodear

Lenguajes:

IoT: Nos conviene codearlo en c++, es lo que se suele usar para el esp32

Por que java + spring? Integracion sencilla con apis, facilidad para mantener y escalar en el futuro, excelente integracion con IoT, persistencia fuerte con Data JPA + Hibernate, Java es excelente para modelar motores de reglas complejo, Java maneja muy bien la concurrencia y estabilidad, fuertemente tipado y tambien codeo en eso!!

Que modelo? YOLOv8 Classification es la que recomienda el **amigo** GPT. ¿Por qué?

- fácil de entrenar
- muy defendible
- rápido
- funciona con datasets chicos
- fácil inferencia local
- puede correr hasta en edge

Ventaja enorme:

mañana podés evolucionar de clasificación → detección.

El problema que surge es la cantidad de imagenes que se necesita (sorry se viene mucho texto pero es para debatir que opinan uds)

La respuesta corta: **no, probablemente no para yerba mate**, y no pasa nada. De hecho, en tesis/MVP de visión artificial agrícola es bastante normal trabajar con datasets chicos.

Tu estrategia no debería ser:

“consigamos miles de imágenes por clase”

sino:

“reduzcamos el problema para que el dataset sea viable”.

Ahí está la diferencia entre un paper ideal y un proyecto final defendible.

Primero: no todas las clases necesitan el mismo volumen

Tus clases actuales son:

- 1 Healthy
- 2 SolarStress
- 3 Chlorosis
- 4 FoliarPest
- 5 StructuralCollapse

Hay clases MUCHO más fáciles de aprender visualmente.

Por ejemplo:

Healthy

Fácil.

Van a tener muchísimas imágenes.

SolarStress (quemadura)

Fácil/moderada.

Visualmente distintiva.

Manchas claras, blanqueamiento, necrosis.

No necesita miles.

Chlorosis

Muy fácil.

El amarillamiento es muy evidente.

StructuralCollapse (colapso)

Muy fácil.

La geometría cambia totalmente.

Plantín caído, marchito.

FoliarPest

La más difícil.

Porque:

- variabilidad visual
- daño parcial
- diferentes síntomas

Acá conviene simplificar muchísimo.

Lo que haría yo: reducir el problema visual

En vez de:

Ácaro

Rulo

Hongo A

Hongo B

Hongo C

hacer:

Healthy

AbioticStress

BioticDamage

Collapse

o mantener tus 5 clases pero agrupar agresivamente.

Tu objetivo no es un paper científico.

Tu objetivo es:

demostrar automatización de lazo cerrado defendible.

El jurado rara vez te va a pedir 99% accuracy.

Va a valorar:

- arquitectura
 - criterio técnico
 - integración
 - viabilidad
-

Con pocas imágenes igual funciona

Acá entra el verdadero truco:

Transfer Learning

Vos NO entrenás desde cero.

El modelo ya viene sabiendo reconocer:

- bordes
- texturas
- patrones visuales
- color
- formas

Vos solo le enseñás:

“esto en yerba mate significa clorosis”

Por eso no necesitás miles.

Un escenario realista:

Clase	Cantidad
Healthy	300
Chlorosis	120
SolarStress	100
FoliarPest	80
Collapse	60

Total:

~650 imágenes.

Eso ya puede ser defendible usando transfer learning.

Y además hacés Data Augmentation

En agricultura esto se usa muchísimo.

De una foto generás varias variaciones:

- rotación
- zoom
- brillo
- contraste
- blur leve
- flip horizontal
- crop

Ejemplo:

1 foto →

original

rotada

más oscura

más brillante

zoom

crop

leve ruido

1 imagen real ⇒ 8–20 muestras de entrenamiento.

Entonces:

650 imágenes reales

↓

augmentation

↓

5000+ samples

Y eso ya empieza a ser bastante usable.

Otra cosa muy importante: no hace falta IA “perfecta”

Esto es clave para defender.

Tu sistema es multimodal.

No depende solo de visión.

Ejemplo:

imagen → posible collapse (0.72)

+

humedad > 80%

+

temperatura alta

=

probable damping off

Eso es MUY fuerte conceptualmente.

Porque el sistema no dice:

“la cámara sabe todo”

dice:

“la cámara aporta evidencia y el motor agronómico fusiona señales”.

Eso es ingeniería seria.

Incluso podés arrancar con un MVP todavía más chico

Te doy una estrategia muy defendible:

Fase 1 (tesis)

Solo 3 clases:

Healthy

Chlorosis

SolarStress

¿Por qué?

Porque son las más fáciles de conseguir y visualizar.

Después argumentás:

“La arquitectura fue diseñada para admitir incorporación incremental de nuevas clases (plagas y enfermedades) a medida que el dataset evolucione”.

Eso le encanta a un jurado.

¿Hay datasets públicos?

Sí, pero no específicos de yerba mate.

Podés bootstrpear con datasets de enfermedades vegetales.

Ejemplo:

Kaggle tiene muchos datasets de hojas enfermas.

También el dataset clásico:

PlantVillage

Tiene miles de imágenes de:

- clorosis
- necrosis
- estrés
- hongos
- plagas

No son yerba mate, PERO sirven para:

1. preentrenamiento conceptual
2. transferencia
3. aumentar robustez

Y después hacés fine tuning con tus pocas imágenes reales de yerba.